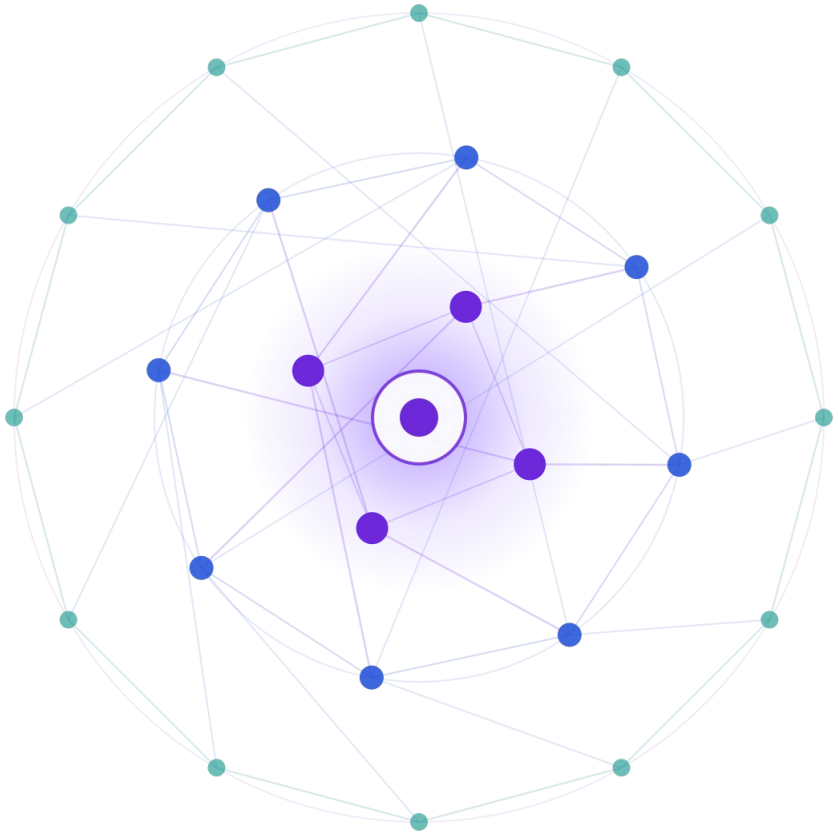
SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27



- Theme ID - Theme 01
- Team Name - Doctors
- College Name - VITV (VIT Vellore)
- Manish H. · manish.h2024@vitstudent.ac.in
- Aryan Sinha · aryan.sinha2024@vitstudent.ac.in
- Munal Singh · munal.singh2024@vitstudent.ac.in
- Anshul Singh · anshul.singh2024@vitstudent.ac.in

Submission GitHub: CodeDoctor repository

Theme

Theme 01: Code intelligence Retrieve ranked code for natural-language queries, with source evidence and support for changing codebases.

Pre/post-processing contribution Reuse unchanged code embeddings before retrieval; verify candidate behavior after retrieval. The embedding model remains fixed.

Evaluation priorities P0: official AppsRetrieval test. P1: rebuild or update after code changes. Bonus: retrieve related variants across versions.

Efficiency guidance The FAQ permits GPU preprocessing and asks for minimal GPU use during retrieval, ideally CPU only. Our selected query path uses CPU.

Existing Solutions & Gaps

Intent and identifiers Lexical search can miss conceptual intent; dense search can miss exact names. The workbench combines both signals.

Repeated indexing cost Unchanged code should not incur repeated model inference. Content and encoder-recipe keys enable vector reuse.

Similar text, different behavior A plausible snippet can fail a worked example. Behavioral checks can improve its ranking when usable examples are present.

Version ambiguity Historical results need explicit snapshot identity. Version isolation prevents a missing version from silently falling back to current code.

Our Solutions & Architecture Diagram

Our Pre/Post-Processing Architecture

1. PREPROCESS (Offline)

- P0: exact CoIR corpus text
- Pinned embedding recipe
- Content-addressed vector cache
- Workbench: AST chunks*
- Reuse unchanged chunks*

2. RETRIEVE (CPU-Only)

- F2LLM-v2-1.7B (accuracy)
- CPU query inference
- Cosine top-20 candidates
- Symbol/BM25 routing*
- Reciprocal rank fusion*

3. POSTPROCESS (Dynamic)

- Parse worked examples
- Execute candidate Python
- Time/resource limits
- Rerank using pass/fail
- Group version variants*

* Workbench and version engine features; they are not all part of the P0 benchmark. P0 uses corpus embeddings, CPU query embeddings and example-based reranking. Verification is conditional on usable examples and is not a correctness proof.

Demo & Product Walkthrough

1. **Index Python code** Open a folder for a working snapshot. The built-in repository demo also exposes its Git history; arbitrary folder targets do not import history.

2. **Query with evidence** Inspect ranked snippets, file/line spans and retrieval signals. Dense queries use the selected encoder tier.

3. **Edit and update** One-function edit: 6.45 s, 2 chunks encoded and 48 reused. Unchanged update: 1.51 s. These are separate measured cases.

4. **Compare variants** A constructed three-version fixture tests lineage grouping and verification. The engine retains version identity for each result.

Recorded walkthrough https://youtu.be/0k2UUJOp-Tc — published recording is 5:46; the supplied template asks for a maximum of 5:00.

Tools and tech stack used

Runtime Python, PyTorch 2.8.0, Sentence Transformers and NumPy. The current retrieval path uses Python; the C++ component is a separate prototype.

Preprocessing Tree-sitter AST chunks, structural metadata and SQLite content-addressed embeddings keyed by the full model recipe.

Retrieval and postprocessing Cosine search; symbol/BM25 and reciprocal rank fusion in the workbench; Python subprocess execution for usable examples.

Embedding models F2LLM-v2-1.7B accuracy tier; 0.6B is explicit opt-in. Revisions, prompts, pooling and sequence limits are pinned.

Evaluation and interface MTEB 2.21.0 with cache=None; pytest and GitHub Actions; FastAPI/Uvicorn workbench. Dependency and source records accompany measurements.

Impact & Use case

Developer onboarding Map a natural-language question to implementation snippets with source locations and structural metadata.

Example-guided triage Use examples from a problem or issue to distinguish candidate behavior, while retaining failures and execution limits.

Changing repositories Reuse unchanged chunks after edits. The measured rebuild fixture demonstrates reuse, rather than a universal update-time guarantee.

Code archaeology Search explicit snapshots and related variants. Version isolation addresses retrieval provenance; it does not establish freedom from pretraining exposure.

Checkable outputs Return ranked source code and retrieval evidence. Code generation and formal proof of correctness are outside this submission.

Innovation highlights, results and limitations

- **P0 — staged CPU query encoding** NDCG@10 0.96583; MRR@10 0.95825; Recall@10 0.98805. Four CPU query shards; official MTEB scoring replay. Prior APPS test exposure; encoding time is separate.
- **CPU P1 — 40 functions, 50 chunks, 3 repetitions** Median full build 102.78 s; unchanged update 1.51 s; one-function edit 6.45 s; ten-function edit 29.51 s. Fresh-index comparison matched all 6 probe rankings.
- **CPU Bonus — constructed 12-lineage fixture** Correct version first: 12/12 with verification, 1/12 without. Median query 5.82 s. These are functional fixture results, not unseen-accuracy estimates.
- **Limits and resources** 4-vCPU Kaggle Xeon; peak RSS: P0 replay 9.41 GB; P1 fixture 10.85 GB. APPS pretraining exposure is possible. Visible examples and subprocess limits do not prove correctness or provide a hardened security boundary.

What’s next

Broader Python inputs Improve handling of incomplete fragments and diverse test formats; measure generalization on genuinely unseen code.

Lower query cost Investigate faster CPU runtimes only when ranking fidelity and end-to-end measurements support adoption. No unmeasured latency target is claimed.

Stronger execution isolation Replace defensive subprocess limits with a hardened isolation boundary for untrusted multi-user deployments.

Richer repository structure Extend cross-file analysis and history ingestion, preserving source locations, snapshot identity and cache-recipe provenance.

Brownie points slide(differentiation)

Our architectural contribution Combine reusable structural preprocessing with conditional behavioral postprocessing around a fixed embedding model.

Selective work Encode changed content and route workbench queries through cheap signals before dense inference when those signals suffice.

Check behavior after retrieval Use executable examples to rerank semantic candidates; keep the dense order when useful verification evidence is unavailable.

Traceable versions and vectors Retain snapshot identity, content hashes and full encoder recipes. Validate dimensions, finite values and exact cache coverage.

Research discipline Preserve negative experiments and historical results; distinguish synthetic functional checks, scored benchmark evidence and measured runtime.

Checklist - Submission materials

- **Source and reproducible setup** CodeDoctor repository; README includes clean-environment, live evaluation and version-engine commands.
- **Dependencies** requirement.txt and requirements-submission.txt select the portable submission stack; measured environments are preserved separately.
- **Presentation and disclosure** VITV_Doctors_Submission.pptx and matching PDF; the official LangAI3.0 AI disclosure form records tool assistance.
- **Benchmark evidence** CPU result JSON, ranked predictions and source records; four shard encoding logs separate from scoring replay. Historical GPU results retain their original identity.
- **Demo and submission tag** Video: https://youtu.be/0k2UUJOp-Tc. Submission tag: PRISM_GENAI_HACKATHON_Y2026. The current public video duration is 5:46.

Thank you

CodeDoctor / VITV / Doctors

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India